مبانی برنامه نویسی

Pointers

Dr. Mehran Safayani

safayani@iut.ac.ir

safayani.iut.ac.ir

https://www.aparat.com/mehran.safayani

https://github.com/safayani/Programming_Basics_course

Department of Electrical and computer engineering, Isfahan university of technology, Isfahan, Iran

What is a Pointer?

- **Definition:** A variable whose value is a **memory address**.
- It “points to” another variable where the actual value is stored.
- **Direct vs. Indirect Reference:**
  - A regular variable **directly** references a value (e.g., int count = 10;).
  - A pointer **indirectly** references a value (it holds the *address* of count).
  - This process is called **indirection**.

```
[Pointer Variable] ----> [Another Variable's Memory Address] ----> [Actual Value]
```

Why Use Pointers?

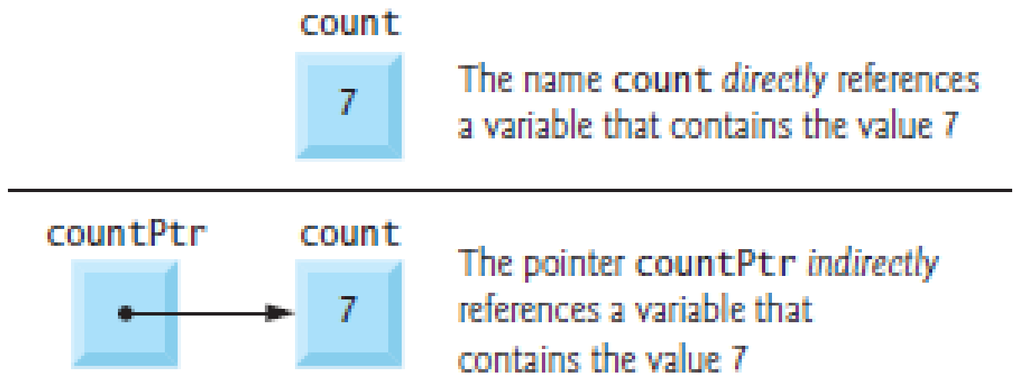
- Pointers are essential for advanced C programming. They enable you to:
- ✅ **Pass-by-reference:** Modify the original arguments of a function.
- ✅ **Manipulate arrays and strings** efficiently.
- ✅ **Pass functions** as arguments to other functions.
- ✅ **Create dynamic data structures:**
  - Linked Lists, Stacks, Queues, Trees.
  - Structures that can grow and shrink during program execution

Declaration and Naming

- Syntax: Pointers must be declared before use. The * indicates it’s a pointer.

```
// Read right-to-left: "countPtr is a pointer to int"
int *countPtr;
```

- **Naming Conventions:** It’s good practice to make pointer variables obvious.
  - End with Ptr (e.g., countPtr)
  - Start with p or p_ (e.g., pCount, p_count)



Declaration Pitfall: A Common Mistake

- The * in a declaration applies **only to the variable immediately following it**.
- **Incorrect Assumption:**

```
// This does NOT declare two pointers!
int *countPtr, count;
```

  - countPtr is a pointer to an int (int *).
  - count is just a regular int
- **Best Practice:** Declare each pointer on its own line to avoid confusion

```
int *countPtr;
int count;
```

Pointer Initialization

- **Crucial:** Always initialize pointers to prevent them from pointing to a random, invalid memory location.
- **Three Ways to Initialize:**
  - **NULL**: A symbolic constant (in <stddef.h>) that means the pointer points to nothing. **This is the preferred method.**

```
int *ptr = NULL;
```

  - **0**: Equivalent to NULL. The value 0 is the only integer that can be directly assigned to a pointer

```
int *ptr = 0;
```

  - **An Address**: Assigning the memory address of another variable (covered in the next section).

The Address Operator: &

- **Purpose:** Gets the **memory address** of its operand.
- **Syntax:** &variableName
- **How it Works:**
  - Given a variable, & returns where it’s located in memory.
  - This address can then be assigned to a pointer.
- **Rule:** The operand of & **must be a variable**. It cannot be a literal value (e.g., &5) or an expression.
-

```
int y = 5;
// Initialize yPtr with the memory address of y
int *yPtr = &y;

// We now say that "yPtr points to y"
```

The Indirection Operator: *

- **Purpose:** Accesses the **value** stored at the address a pointer is holding.
- **Also called:** The **Dereferencing Operator**.
- **Syntax:** *pointerName
- **How it Works:**
  - It follows the pointer to its stored memory address and retrieves the data from that location.

```
// yPtr holds the address of y.
// *yPtr accesses the value at that address.
printf("%d", *yPtr); // This will print 5
```

CRITICAL WARNING: Dereferencing Uninitialized Pointers

- Accessing the value of a pointer that doesn’t point to a valid memory location is a **major error**.
- **Never dereference a pointer that has not been initialized!**
- **Potential Consequences:**
  - **Fatal Crash:** Your program may stop working immediately.
  - **Data Corruption:** It could accidentally modify important data, leading to incorrect results that are hard to debug.
  - **Security Breaches:** Can create vulnerabilities that could be exploited.

& and * are Complements

- The address (&) and indirection (*) operators are inverses of each other.
- Given int y = 5; and int *yPtr = &y;
- Applying * to a pointer **gets the original value**.
  - *yPtr is equivalent to y (both are 5).
- Applying & to a dereferenced pointer **gets the original address**.
  - &(*yPtr) is equivalent to &y.
- They effectively “cancel each other out.”

Practical Tip: Printing Addresses

- To see the memory addresses that pointers store, use the %p conversion specifier in printf.
- It displays the memory address in a system-specific format (usually hexadecimal).

```c
int a = 7;
int *aPtr = &a;

printf("The address of a is %p\n", &a);
printf("The value of aPtr is %p\n", aPtr);
```

```
The address of a is 0x7ffc1a3b4c54
The value of aPtr is 0x7ffc1a3b4c54
```

```c
// fig07_01.c
// Using the & and * pointer operators.
#include <stdio.h>

int main(void) {

   int a = 7;
   int *aPtr = &a; // set aPtr to the address of a

   printf("Address of a is %p\nValue of aPtr is %p\n\n", &a, aPtr);
   printf("Value of a is %d\nValue of *aPtr is %d\n\n", a, *aPtr);
   printf("Showing that * and & are complements of each other\n");
   printf("&*aPtr = %p\n*&aPtr = %p\n", &*aPtr, *&aPtr);
}
```

```
Address of a is 0x7fffe69386cc
Value of aPtr is 0x7fffe69386cc

Value of a is 7
Value of *aPtr is 7

Showing that * and & are complements of each other
&*aPtr = 0x7fffe69386cc
*&aPtr = 0x7fffe69386cc
```

Fig. 7.1 | Using the & and * pointer operators. (Part 2 of 2.)

Key Takeaways

- **& (Address Operator):** Gets the **address** of a variable.
  - &y → “Where does y live in memory?”
- *** (Indirection Operator):** Gets the **value** at an address.
  - *yPtr → “What is the value at the location yPtr points to?”
- **Golden Rule: ALWAYS** initialize a pointer before you dereference (*) it.
- The %p format specifier is used to print pointer addresses.

Operators	Grouping	Type
() [] ++ *(postfix)* -- *(postfix)*	left to right	postfix
+ - ++ -- ! * & (*type*)	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > s>=	left to right	relational
== !=	left to right	equality
&&	left to right	logical AND
||	left to right	logical OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment
,	left to right	comma

Passing Arguments to Functions in C

- **Pass-by-Value (The Default)**
  - The function receives a **copy** of the argument's value.
  - The original variable in the calling function is **unaffected** and cannot be changed.
  - Safe and prevents accidental side effects.
- **Pass-by-Reference**
  - The function receives the **memory address** of the argument.
  - The function can directly access and **modify** the original variable.
  - Achieved using pointers (& and *).

How Pass-by-Value Works

- A temporary copy of the variable is created inside the function.
- All operations are performed on this local copy.
- When the function ends, the copy is destroyed.
- **The original variable remains unchanged.**

```
void cubeByValue(int n) {
    n = n * n * n; // Modifies the local copy 'n'
}

// In main():
int number = 5;
cubeByValue(number);
// 'number' is STILL 5 here.
```

How Pass-by-Reference Works

- This is a 3-step process using pointer operators:
  - **Caller:** Pass the **address** of the variable using the & operator.
  - **Function Definition:** The parameter must be a **pointer** (using *) to receive the address.
  - **Inside Function:** Use the **dereference operator (*)** to access and modify the original value.

```
void cubeByReference(int *nPtr) {
    *nPtr = (*nPtr) * (*nPtr) * (*nPtr); // Modifies original value
}

// In main():
int number = 5;
cubeByReference(&number); // Step 1: Pass the address
// 'number' is now 125!
```

Why Use Pass-by-Reference?

- It provides capabilities that pass-by-value cannot:
- ✅ **Modify Caller’s Data:** The primary reason. Allows a function to make lasting changes to variables outside its own scope.
- ✅ **“Return” Multiple Values:** A function can only have one return statement, but by accepting multiple pointers, it can modify several variables in the caller.
- ✅ **Efficiency:** Avoids the performance overhead of copying large data structures (like structs). Passing a small address is much faster than copying a large object.
-

Special Case: Arrays

- In C, arrays are **always passed by reference** by default.
- The name of an array is essentially a pointer to its first element (&arrayName[0]).
- You **do not** use the & operator when passing an array.
- **Compiler Interchangeability:** In a function's parameter list, the compiler treats int arr[] and int *arr as identical.

```
// These two function prototypes are seen as the same by the compiler:
void processArray(int arr[], int size);
void processArray(int *arr, int size);
```

Summary and Best Practices

Feature	Pass-by-Value	Pass-by-Reference
What is Passed?	A **copy** of the value	The **memory address** of the variable
Can Original Be Modified?	**No**	**Yes**
How to Call	myFunc(variable);	myFunc(&variable);
Function Parameter	void myFunc(int n);	void myFunc(int *nPtr);
Use When…	You only need to use the value.	You need to change the original variable.

Principle of Least Privilege: Use **pass-by-value** as your default. Only use pass-by-reference when the function is specifically required to modify the caller's data. This makes your code safer and easier to understand

Why Use the const Qualifier?

- const tells the compiler that a value should not be modified.
- **Enforces Least Privilege:** A function gets only the permissions it needs, and no more.
- **Reduces Bugs:** The compiler catches accidental modifications at compile-time, preventing runtime errors and unintentional side effects.
- **Improves Maintainability:** Makes code easier to understand and safer to modify.
- **A Key to Modernizing Legacy Code:** Many older C programs can be improved by correctly adding const.

The Four Combinations of const and Pointers

- There are four ways to use const with a pointer, each granting different permissions.
- **Non-Constant Pointer to Non-Constant Data** (Most permissions)
  - int *ptr;
- **Non-Constant Pointer to Constant Data**
  - const int *ptr;
- **Constant Pointer to Non-Constant Data**
  - int * const ptr;
- **Constant Pointer to Constant Data** (Least permissions)
  - const int * const ptr;

Type 1: Non-Constant Pointer to Non-Constant Data

- char *sPtr;
- **What it means:** The highest level of access.
- **Can the pointer be changed to point elsewhere?** ✅ Yes. (sPtr++)
- **Can the data it points to be modified?** ✅ Yes. (*sPtr = 'A')
- **Use Case:** When a function needs to both modify the data and iterate through an array by moving the pointer.
  - *Example from text: A function convertToUppercase that changes characters in place.*

```c
// fig07_06.c
// Converting a string to uppercase using a
// non-constant pointer to non-constant data.
#include <ctype.h>
#include <stdio.h>

void convertToUppercase(char *sPtr); // prototype

int main(void) {
   char string[] = "cHaRaCters and $32.98"; // initialize char array

   printf("The string before conversion is: %s\n", string);
   convertToUppercase(string);
   printf("The string after conversion is: %s\n", string);
}

// convert string to uppercase letters
void convertToUppercase(char *sPtr) {
   while (*sPtr != ) { // current character is not
      *sPtr = toupper(*sPtr); // convert to uppercase
      ++sPtr; // make sPtr point to the next character
   }
}
```

```
The string before conversion is: cHaRaCters and $32.98
The string after conversion is: CHARACTERS AND $32.98
```

Fig. 7.6 | Converting a string to uppercase using a non-constant pointer to non-constant data.

Type 2: Non-Constant Pointer to Constant Data

- const char *sPtr; (Read right-to-left: “sPtr is a pointer to a char that is const”)
- **What it means:** The pointer can move, but the data it points to is read-only.
- **Can the pointer be changed to point elsewhere?** ✅ Yes. (sPtr++)
- **Can the data it points to be modified?** ❌ **No.** (*sPtr = 'A' -> **COMPILER ERROR**)
- **Use Case:** Safely reading data (like printing a string) without any risk of changing it.
  - *This is also used to pass large structs efficiently (by reference) while getting the safety of pass-by-value.*

```c
// fig07_07.c
// Printing a string one character at a time using
// a non-constant pointer to constant data.

#include <stdio.h>

void printCharacters(const char *sPtr);

int main(void) {
   // initialize char array
   char string[] = "print characters of a string";

   puts("The string is:");
   printCharacters(string);
   puts("");
}

// sPtr cannot be used to modify the character to which it points,
// i.e., sPtr is a "read-only" pointer
void printCharacters(const char *sPtr) {
   // loop through entire string
   for (; *sPtr != ; ++sPtr) { // no initialization
      printf("%c", *sPtr);
   }
}
```

```
The string is:
print characters of a string
```

Fig. 7.7 | Printing a string one character at a time using a non-constant pointer to constant data.

```c
// fig07_08.c
// Attempting to modify data through a
// non-constant pointer to constant data.
#include <stdio.h>
void f(const int *xPtr); // prototype

int main(void) {
   int y = 7; // define y

   f(&y); // f attempts illegal modification
}

// xPtr cannot be used to modify the
// value of the variable to which it points
void f(const int *xPtr) {
   *xPtr = 100; // error: cannot modify a const object
}
```

```
fig07_08.c(16,5): error C2166: l-value specifies const object
```

Fig. 7.8 | Attempting to modify data through a non-constant pointer to constant data.

Type 3: Constant Pointer to Non-Constant Data

- int * const ptr; (Read right-to-left: “ptr is a const pointer to an int”)
- **What it means:** The pointer is locked to one memory address, but the data at that location can be changed.
- **Can the pointer be changed to point elsewhere?** ❌ **No.** (ptr = &y -> **COMPILER ERROR**)
- **Can the data it points to be modified?** ✅ Yes. (*ptr = 100)
- **Use Case:** When a pointer must always refer to the same variable, like an array name. Must be initialized when declared.

```c
// fig07_09.c
// Attempting to modify a constant pointer to non-constant data.
#include <stdio.h>

int main(void) {
   int x = 0; // define x
   int y = 0; // define y

   // ptr is a constant pointer to an integer that can be modified
   // through ptr, but ptr always points to the same memory location
   int * const ptr = &x;

   *ptr = 7; // allowed: *ptr is not const
   ptr = &y; // error: ptr is const; cannot assign new address
}
```

```
fig07_09.c(14,4): error C2166: l-value specifies const object
```

Fig. 7.9 | Attempting to modify a constant pointer to non-constant data.

Type 4: Constant Pointer to Constant Data

- const int * const ptr; (Read right-to-left: “ptr is a const pointer to an int that is const”)
- **What it means:** The most restrictive. A read-only pointer that points to a read-only location.
- **Can the pointer be changed to point elsewhere? ❌ No.** (ptr = &y -> **COMPILER ERROR**)
- **Can the data it points to be modified? ❌ No.** (*ptr = 100 -> **COMPILER ERROR**)
- **Use Case:** For maximum safety, when a function should only read a value from a fixed memory location (e.g., a hardware register or a fixed lookup table).

```c

int main(void) {
   int x = 5;
   int y = 0;

   // ptr is a constant pointer to a constant integer. ptr always
   // points to the same location; the integer at that location
   // cannot be modified
   const int *const ptr = &x; // initialization is OK

   printf("%d\n", *ptr);
   *ptr = 7; // error: *ptr is const; cannot assign new value
   ptr = &y; // error: ptr is const; cannot assign new address
}
```

```
fig07_10.c(15,5): error C2166: l-value specifies const object
fig07_10.c(16,4): error C2166: l-value specifies const object
```

Fig. 7.10 | Attempting to modify a constant pointer to constant data. (Part 2 of 2.)

Summary & Guiding Principle

- **Let the Principle of Least Privilege be your guide.**
- Always give a function *just enough* access to do its job, and absolutely no more.

If a function needs to…	Use this parameter type:	Pointer Movable?	Data Modifiable?
Modify an array’s contents	int * ptr	✅	✅
Read an array’s contents without changing them	const int * ptr	✅	❌
Modify a single, fixed variable via a pointer	int * const ptr	❌	✅
Read a single, fixed variable via a pointer	const int * const ptr	❌	❌

Bubble Sort with Pass-by-Reference

- The objective is to refactor the Bubble Sort algorithm into two distinct functions:
- **bubbleSort(array, size)**
  - Contains the main sorting logic (the loops).
- **swap(pointer1, pointer2)**
  - A helper function responsible for only one thing: exchanging two values.
- This separation of concerns makes the code cleaner and more modular.

The Challenge: Modifying Array Elements

- **Problem:** C enforces **information hiding**. The swap function does not have direct access to the array variable inside bubbleSort.
- Individual array elements (e.g., array[j]) are simple values (scalars). By default, they are **passed-by-value**.
- A copy of array[j] would be sent to swap, and swapping copies would have **no effect** on the original array.
- **How do we allow swap to modify the actual array elements?**

The Solution: Pass-by-Reference for Elements

- We use pointers to give swap access to the original data.
- **In bubbleSort:** We pass the **memory address** of each element using the & operator.

```
// Pass the addresses of the two elements to be swapped
swap(&array[j], &array[j + 1]);
```

- **In swap:** The function parameters are defined as **pointers** to receive these addresses.

```
void swap(int *element1Ptr, int *element2Ptr) { ... }
```

- Inside swap, *element1Ptr now becomes a synonym for the original array[j].

Inside the swap Function Logic

- By dereferencing the pointers, swap can now modify the values in the original array.
- Even though swap cannot access array[j] by name…
- …it achieves the exact same effect by using the pointers it received.

```
// This code modifies the original array elements in bubbleSort
void swap(int *element1Ptr, int *element2Ptr) {
    int hold = *element1Ptr;
    *element1Ptr = *element2Ptr;
    *element2Ptr = hold;
}
```

Software Engineering Principle 1: Reusability

- **Why must we pass the array size to bubbleSort?**
- When you pass an array to a function, you are only passing the memory address of its first element. The address itself contains **no information about the array's length**.
- **Passing the size makes the function reusable.** bubbleSort can now work on an integer array of *any* size.
- **Alternatives are Bad Practice:**
  - **Hard-coding the size:** Makes the function useless for any other array size.
  - **Using a global variable:** Violates the principle of least privilege and prevents the function from being easily used in other programs.

Software Engineering Principle 2: Least Privileg

- **Why is the prototype for swap placed inside bubbleSort's body?**

```
void bubbleSort(int * const array, const size_t size) {
    void swap(int *element1Ptr, int *element2Ptr); // Prototype is here!
    // ... loops and calls to swap() ...
}
```

- This is a design choice that enforces the **Principle of Least Privilege**.
- By placing the prototype locally, swap is only "visible" and callable from within bubbleSort.
- This prevents other functions in the program from accidentally (or intentionally) calling a helper function that they are not supposed to use.

```c
// fig07_11.c
// Putting values into an array, sorting the values into
// ascending order and printing the resulting array.
#include <stdio.h>
#define SIZE 10

void bubbleSort(int * const array, size_t size); // prototype

int main(void) {
   // initialize array a
   int a[SIZE] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };

   puts("Data items in original order");

   // loop through array a
   for (size_t i = 0; i < SIZE; ++i) {
      printf("%4d", a[i]);
   }

   bubbleSort(a, SIZE); // sort the array

   puts("\nData items in ascending order");

   // loop through array a
   for (size_t i = 0; i < SIZE; ++i) {
      printf("%4d", a[i]);
   }

   puts("");
}
```

```
32  // sort an array of integers using bubble sort algorithm
33  void bubbleSort(int * const array, size_t size) {
34     void swap(int *element1Ptr, int *element2Ptr); // prototype
35
36     // loop to control passes
37     for (int pass = 0; pass < size - 1; ++pass) {
38        // loop to control comparisons during each pass
39        for (size_t j = 0; j < size - 1; ++j) {
40           // swap adjacent elements if they're out of order
41           if (array[j] > array[j + 1]) {
42              swap(&array[j], &array[j + 1]);
43           }
44        }
45     }
46  }

48  // swap values at memory locations to which element1Ptr and
49  // element2Ptr point
50  void swap(int *element1Ptr, int *element2Ptr) {
51     int hold = *element1Ptr;
52     *element1Ptr = *element2Ptr;
53     *element2Ptr = hold;
54  }
```

```
Data items in original order
   2   6   4   8  10  12  89  68  45  37
Data items in ascending order
   2   4   6   8  10  12  37  45  68  89
```

Fig. 7.11 | Putting values into an array, sorting the values into ascending order and printing the resulting array. (Part 2 of 2.)

What is the sizeof Operator?

- A **unary operator** that determines the size of its operand in **bytes**.
- The result is a size_t value.
- It’s a **compile-time** operator, meaning it’s resolved during compilation and has no performance cost when the program runs.
  - *(Exception: Variable-Length Arrays, VLAs)*
- **Syntax:**
  - With a type: sizeof(int) (Parentheses are **required**)
  - With a variable: sizeof array or sizeof(array) (Parentheses are optional)

sizeof with Arrays

- This is a primary and powerful use of the operator.
- When applied to an array name, sizeof returns the **total number of bytes occupied by the entire array**.

```c
// A float is typically 4 bytes
float prices[20];

// This calculates the total size: 20 elements * 4 bytes/element
size_t total_bytes = sizeof(prices);

printf("Total size of array is: %zu bytes\n", total_bytes);
```

```
Total size of array is: 80 bytes
```

CRITICAL PITFALL: sizeof with Pointers

- This is the most common point of confusion when using sizeof.
  - When an array is passed to a function, what the function actually receives is a **pointer** to the array's first element.
  - Applying sizeof to that pointer **returns the size of the pointer variable itself**, NOT the size of the original array.

```c
// This function will NOT return the array's size
void getSize(int *ptr) {
    printf("Size inside function: %zu\n", sizeof(ptr));
}

int main() {
    int numbers[20]; // sizeof(numbers) here is 20 * 4 = 80
    getSize(numbers);
}
```

```
Size inside function: 8  // On a 64-bit system where pointers are 8 bytes
```

The “Number of Elements” Idiom

- You can use sizeof to calculate the number of elements in an array.
  - **The Formula:** sizeof(arrayName) / sizeof(arrayName[0])
  - **How it Works:**
    - It divides the total bytes of the array by the number of bytes in a single element.

```c
double measurements[22]; // A double is typically 8 bytes

// (22 * 8) / 8 = 176 / 8 = 22
size_t count = sizeof(measurements) / sizeof(measurements[0]);

printf("The array has %zu elements.\n", count);
```

```
The array has 22 elements.
```

Portability and General Usage

- **Problem:** The size of data types (int, long, pointers, etc.) can vary across different systems and compilers (e.g., 32-bit vs. 64-bit machines).
- **Solution:** Use sizeof to write **portable code** that doesn't depend on hard-coded, assumed sizes. This makes your programs more robust and less likely to fail when compiled on a different machine.
-

```c
// This code will always show the correct sizes for the system it's compiled on
printf("Size of int: %zu bytes\n", sizeof(int));
printf("Size of double: %zu bytes\n", sizeof(double));
printf("Size of a pointer: %zu bytes\n", sizeof(int*));
```

Key Takeaways

- sizeof returns the size of a variable or type in **bytes**.
- sizeof(array) -> Gives the **total size** of the array.
- sizeof(pointer) -> Gives the size of the **pointer variable** itself (typically 4 or 8 bytes).
- To find the number of elements in an array, use: sizeof(array) / sizeof(array[0])
- Rely on sizeof to make your code portable across different computer systems.

Valid Pointer Arithmetic Operations

- Not all standard math operators are valid with pointers. Pointer arithmetic is specifically designed for navigating arrays.
- The allowed operations are:
- ✅ **Incrementing (++)** and **Decrementing (--)** a pointer.
- ✅ **Adding an integer** to a pointer (+ or +=).
- ✅ **Subtracting an integer** from a pointer (- or -=).
- ✅ **Subtracting two pointers** from each other (if they point to the same array).
- ⚠️ **Rule:** These operations are only meaningful when the pointers point to elements within the same array.

The Core Concept: Scaled by Type Size

- Pointer arithmetic is **not** conventional arithmetic.
- When you add an integer N to a pointer, the address is increased by: N * sizeof(the_type_the_pointer_points_to)
- **Example:** Assume int is 4 bytes.

```
int v[5];        // Array of integers
int *vPtr = v;   // vPtr points to v[0] at address 3000

vPtr += 2;       // This does NOT result in 3002!
```

- **The Calculation:** 3000 + (2 * sizeof(int)) -> 3000 + (2 * 4) -> 3008
- The pointer vPtr now correctly points to the element v[2].

Pointer Operations in Practice

- Given int *vPtr pointing to v[0] (address 3000):
- **Incrementing (++):** vPtr++;
  - Moves the pointer to the *next element* (v[1] at address 3004).
- **Decrementing (--):** vPtr--;
  - Moves the pointer to the *previous element*.
- **Adding an Integer:** vPtr = vPtr + 3;
  - Moves the pointer forward 3 elements (v[3] at address 3012).
- **Subtracting an Integer:** vPtr = vPtr - 1;
  - Moves the pointer back 1 element

Subtracting Two Pointers

- This operation reveals the distance between two elements.
- When you subtract two pointers (that point to the same array), the result is **not the difference in bytes**, but the **number of elements** between them.

```
int v[5];
int *vPtr1 = &v[0]; // Address is 3000
int *vPtr2 = &v[2]; // Address is 3008

int x = vPtr2 - vPtr1;
```

- The value of x will be **2**, because there are two int elements between the pointers.
- The raw byte difference (3008 - 3000 = 8) is automatically scaled down by sizeof(int).

Assignment and Comparison

- **Assignment (=):**
  - A pointer can be assigned the value of another pointer of the same type.
- **Comparison (==, !=, <, >):**
  - Pointers can be compared, but it’s only meaningful if they point to elements of the **same array**.
  - This is useful for checking if a pointer is at the beginning or end of an array.
  - A very common comparison is checking if a pointer is NULL. if (ptr == NULL) { ... }
-

The Generic Pointer: void *

- **What it is:** A void * is a generic pointer that can hold the address of **any data type**.
- **Assignment:** You can assign any pointer type to a void * (and vice-versa) without an explicit cast.

```
int x = 10;
void *genericPtr = &x; // Perfectly valid
```

- **CRITICAL LIMITATION:** You **cannot dereference** a void *.

*genericPtr = 20; // COMPILER ERROR

  - **Why?** The compiler doesn't know the type (and therefore the size) of the data it points to. It doesn't know if it should modify 1, 4, or 8 bytes of memory.